

String

- String is **not a primitive type**; it is a **class/object type** in Java.
- A String stores a **sequence of characters**.
- Strings are written inside **double quotes** (" ").

```
String motto = "String is a sequence of chars.";
```

Here:

- String → data type (class)
- motto → variable
- "String is a sequence of chars." → String object/value

Concatenation of Strings

- Using + operator (most common)

```
String first = "Hello";
String second = "World";
String result = first + " " + second; // "Hello World"
```

```
first + 2026; // "Hello2026"
2026 + first; // "2026Hello"
```

- Using String.concat()

```
String a = "Java";
String b = " Programming";
String result = a.concat(b); // "Java Programming"
```

String methods

1.. length()

Definition:

```
int length()
```

- Returns the number of characters in the string.

Example:

```
String text = "Programming";
int len = text.length(); // len = 11
len = "Hi".length();
```

2. equals()

Definition:

```
boolean equals(String other)
```

- Compares the content of the calling string with another string.

Example:

```
String greeting = "Hello";  
greeting.equals("Hello");//true
```

3. charAt ()

Definition:

```
char charAt(int index);
```

- Returns the character at a specific index (starting at 0).

Example:

```
String word = "Java";  
char ch = word.charAt(2); // ch = 'v'
```

4. compareTo ()

```
int compareTo(String anotherString)
```

Compares two strings lexicographically (dictionary order) using Unicode values of characters.

- Return values

< 0 → calling string comes before anotherString

0 → strings are equal

> 0 → calling string comes after anotherString

```
String a = "adventure";
```

```
a.compareTo("zoo");           // negative  
a.compareTo("adventure");     // 0  
a.compareTo("above");         // positive
```

Method	Use Case (What it's used for)	Example Code	Result
length()	Find number of characters in a string	"Hello!".length()	6
equals(String s)	Check if two strings are exactly the same (case-sensitive)	"Hello".equals("Hello")	true
equalsIgnoreCase(String s)	Compare strings ignoring case	"mary".equalsIgnoreCase("Mary")	true
toLowerCase()	Convert all letters to lowercase	"Hi Mary!".toLowerCase()	"hi mary!"
toUpperCase()	Convert all letters to uppercase	"Hi Mary!".toUpperCase()	"HI MARY!"
trim()	Remove leading & trailing spaces	" Hmm ".trim()	"Hmm"
charAt(int i)	Get character at a specific index	"Hello".charAt(1)	'e'

Method	Use Case (What it's used for)	Example Code	Result
<code>substring(int start)</code>	Extract substring from index to end	<code>"AbcdefG".substring(2)</code>	"cdefG"
<code>substring(int start, int end)</code>	Extract substring between two indexes	<code>"AbcdefG".substring(2,5)</code>	"cde"
<code>indexOf(String s)</code>	Find first occurrence of substring	<code>"Hi Mary!".indexOf("Mary")</code>	3
<code>indexOf(String s, int start)</code>	Find substring starting from index	<code>"Mary, Mary".indexOf("Mary",1)</code>	6
<code>lastIndexOf(String s)</code>	Find last occurrence of substring	<code>"Mary, Mary".lastIndexOf("Mary")</code>	6
<code>compareTo(String s)</code>	Alphabetical comparison (case-sensitive)	<code>"apple".compareTo("banana")</code>	Negative
<code>compareToIgnoreCase(String s)</code>	Alphabetical comparison ignoring case	<code>"Apple".compareToIgnoreCase("apple")</code>	0

Full method list: [Java String API](#)